

Relatorio - AP Project 2: Parallel Galaxy Simulation

Disciplina: Programacao Avancada - Modulo de Computacao Paralela Universidade: Universidade da Madeira Ano: 2025/2026 Grupo: substituir por ID_ID_ID_ID

1. Objetivo

O objetivo deste projeto e implementar uma simulacao paralela de uma galaxia usando Python e mpi4py. A galaxia e representada por particulas, onde cada particula corresponde a uma estrela com posicao, velocidade e massa. Em cada passo temporal, cada processo MPI calcula as forcas gravitacionais das estrelas que lhe foram atribuidas, atualiza as velocidades e posicoes, e periodicamente envia os dados para o processo 0 para produzir graficos 2D e 3D e uma animacao GIF.

A ficha do projeto pede uma simulacao com Newton, comunicacao MPI, medicao de tempo, plots 2D/3D e GIF. Tambem indica que o alvo teorico e 10^9 particulas e 10^9 anos simulados, mas exige explicar as limitacoes de um metodo direto $O(N^2)$.

2. Modelo fisico: Segunda Lei de Newton

A simulacao usa a Segunda Lei de Newton:

$$F = m * a$$

Para cada particula i , a aceleracao e calculada a partir da soma das contribuicoes gravitacionais de todas as outras particulas j :

$$a_i = G * \sum(j \neq i) [m_j * (r_j - r_i) / ((|r_j - r_i|^2 + \epsilon^2)^{3/2})]$$

Onde:

- r_i e a posicao da particula i ;
- m_j e a massa da particula j ;
- G e a constante gravitacional em unidades normalizadas;
- ϵ e um fator de suavizacao para evitar divisoes por zero e aceleracoes infinitas quando duas particulas ficam demasiado proximas.

Depois de calcular a aceleracao, a velocidade e a posicao sao atualizadas com o metodo de Euler semi-implicito:

$$v_i(t + dt) = v_i(t) + a_i(t) * dt$$

$$r_i(t + dt) = r_i(t) + v_i(t + dt) * dt$$

Este metodo e simples e suficiente para demonstrar a paralelizacao. Para simulacoes fisicas mais precisas, seria melhor usar integradores como Leapfrog/Verlet.

3. Distribuicao inicial das particulas

O processo 0 cria um plano de distribuicao com o numero de particulas, offset global e seed aleatoria para cada rank. Esse plano e enviado com `scatter()`. Depois, cada processo gera localmente as suas particulas, o que evita que o rank 0 tenha de criar toda a galaxia sozinho.

A distribuicao inicial usa:

- raio com distribuicao exponencial, para concentrar mais estrelas perto do centro;

- angulo uniforme entre 0 e 2π ;
- coordenada z com distribuicao normal, para criar uma espessura pequena no disco;
- massa com distribuicao lognormal;
- velocidade inicial aproximadamente tangencial, para criar movimento de rotacao.

Esta estrategia e mais escalavel do que gerar todas as particulas no processo 0, porque divide tanto a memoria como o custo de geracao inicial.

4. Estrategia paralela

A paralelizacao segue um modelo de decomposicao por dados. Se houver N particulas e P processos, cada rank fica responsavel por cerca de N/P particulas. Cada processo calcula apenas a aceleracao e a atualizacao das suas particulas locais.

Em cada passo temporal, todas as posicoes e massas sao necessarias para calcular a interacao gravitacional direta. Por isso, cada rank envia o seu estado local e recebe o estado global usando `allgather()`. Em seguida, cada rank calcula as interacoes das suas particulas locais contra todas as particulas globais.

O custo computacional por passo e aproximadamente:

$$O(N^2 / P)$$

O custo de comunicacao por passo e elevado, porque o estado global tem de ser sincronizado entre todos os processos. Isto e aceitavel para desenvolvimento e demonstracao, mas nao e adequado para 10^9 particulas.

5. Metodos MPI utilizados

Foram usados os seguintes metodos MPI:

Metodo MPI	Onde e usado	Justificacao
<code>bcast()</code>	Envio dos parametros de simulacao para todos os ranks	Todos os processos precisam dos mesmos parametros fisicos e temporais.
<code>scatter()</code>	Distribuicao do plano de trabalho inicial	Cada rank recebe o seu numero de particulas, offset e seed.
<code>isend() / irecv()</code>	Diagnostico de balanceamento de carga entre ranks vizinhos	Demonstra comunicacao ponto-a-ponto nao bloqueante sem parar a simulacao.
<code>allgather()</code>	Sincronizacao do estado global em cada passo	Cada rank precisa das posicoes/massas globais para calcular as forcas.
<code>gather()</code>	Recolha de dados para plotting no rank 0	Apenas o rank 0 cria frames e o GIF.
<code>reduce()</code>	Soma da energia cinetica e reducao do tempo maximo de execucao	Permite obter estatisticas globais de forma correta.
<code>Barrier()</code>	Antes/depois da medicao de tempo	Garante que a medicao inclui a execucao paralela de forma sincronizada.

6. Sincronizacao

A principal sincronizacao ocorre com `allgather()`, porque todos os processos precisam de uma visao consistente das posicoes e massas antes de calcular as aceleracoes. A barreira `Barrier()` e usada antes de iniciar o cronometro e no final, para medir o tempo total de forma justa. Para gerar frames, `gather()` sincroniza os dados no processo 0, que cria os subplots 2D e 3D.

7. Visualizacao

A simulacao gera um frame no ano 0 e depois a cada 1000 anos simulados, conforme pedido. Cada frame contem:

- subplot 2D: plano x-y da galaxia;
- subplot 3D: coordenadas x, y, z.

No fim, o rank 0 combina todos os frames num ficheiro GIF com `imageio`.

8. Medicao de tempo

O codigo mede o tempo com `MPI.Wtime()`. A medicao comeca depois da distribuicao inicial e termina apos o ultimo passo de simulacao. O tempo reportado e o maximo entre os ranks, obtido com `reduce(..., op=MPI.MAX)`, porque o tempo total de um programa paralelo e limitado pelo processo mais lento.

Tabela para preencher com medicoes reais apos correr no computador/laboratorio:

Particulas	Processos MPI	Anos	dt	Tempo total (s)	Observacoes
400	1	5000	20	preencher	execucao base
400	2	5000	20	preencher	comparar speedup
400	4	5000	20	preencher	comparar speedup
800	1	5000	20	preencher	execucao maior
800	2	5000	20	preencher	comparar speedup
800	4	5000	20	preencher	comparar speedup

Comandos sugeridos:

```
mpiexec -n 1 python galaxy_mpi.py --particles 400 --years 5000 --dt 20
mpiexec -n 2 python galaxy_mpi.py --particles 400 --years 5000 --dt 20
mpiexec -n 4 python galaxy_mpi.py --particles 400 --years 5000 --dt 20
```

9. Bottlenecks principais

O principal bottleneck e o calculo direto das interacoes entre todas as particulas. Para N particulas, cada passo precisa de aproximadamente $N(N-1)$ interacoes, ou seja, complexidade $O(N^2)$. Mesmo com paralelizacao, para 10^9 particulas o custo seria astronomico.

Outro bottleneck e a comunicacao. Como cada rank precisa do estado global em cada passo, `allgather()` envia uma grande quantidade de dados entre processos. Para muitos processos e muitos corpos, a comunicacao pode tornar-se comparavel ou superior ao tempo de computacao.

Tambem ha custo de memoria: guardar posicao, velocidade, massa e ids para 10^9 particulas exige dezenas de gigabytes, mesmo antes de considerar buffers de comunicacao, frames e estruturas auxiliares.

10. O que mudaria para uma simulacao real com 10^9 particulas

Para tornar a simulacao realista nessa escala, seria necessario substituir o metodo direto por algoritmos aproximados ou hierarquicos, por exemplo:

- Barnes-Hut, com complexidade aproximada $O(N \log N)$;
- Fast Multipole Method, que pode aproximar $O(N)$ em certos cenarios;
- decomposicao espacial em vez de distribuicao simples por blocos;
- comunicacao apenas entre regioes vizinhas ou celulas relevantes;
- uso de GPUs para acelerar o calculo das forcas;
- escrita de frames com amostragem, porque guardar um plot a cada 1000 anos durante 10^9 anos produziria cerca de 1 000 000 frames;
- formatos binarios eficientes para checkpoints, em vez de guardar grandes estruturas Python.

Assim, o codigo entregue deve ser visto como uma implementacao correta e paralela do modelo direto para tamanhos pequenos/medios, acompanhada de uma discussao clara sobre porque nao e pratico aplicar diretamente a 10^9 particulas.

11. Contribuicao dos membros do grupo

Preencher antes da entrega:

Membro	Numero de aluno	Contribuicao
Nome 1	ID	Implementacao MPI / testes
Nome 2	ID	Modelo fisico / relatorio
Nome 3	ID	Visualizacao / GIF
Nome 4	ID	Analise de desempenho

12. Conclusao

O projeto implementa uma simulacao paralela de galaxia em Python com `mpi4py`. A solucao usa decomposicao por dados, comunicacao coletiva para sincronizar o estado global, calculo local das aceleracoes e recolha dos dados no rank 0 para visualizacao. A implementacao cumpre o objetivo pedagogico de demonstrar computacao paralela com MPI, mas tambem mostra que o metodo direto $O(N^2)$ nao e escalavel para o alvo teorico de 10^9 particulas.